

16강 상속





1. 상속이란?

- ✓ 클래스를 정의 할 때 이미 구현된 클래스를 **상속(inheritance)** 받아서 속성이나 기능이 확장되는 클래스를 구현함을 말한다.

2. 상속이유

- ✓ 상속의 장점은 코드가 중복되는 것을 막을 수 있고 **공통적인(공유) 코드** 를 변경할 시 시간을 단축 시킬 수 있다.
- ✓ 코드를 재사용한다고 말할 수 있다. 물론 상속이 무조건 코드의 재사용은 아니라는 점은 주의하자.

3. 상속의 클래스 종류

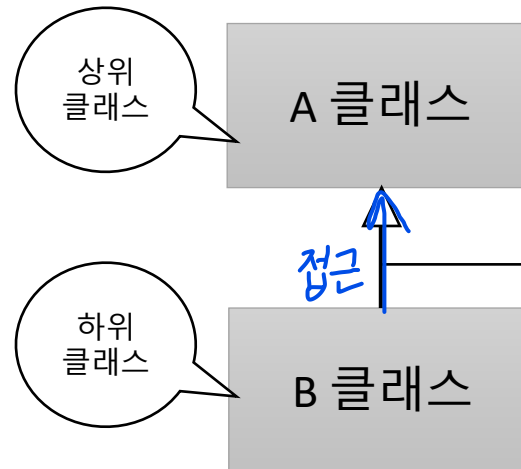
- ✓ 상속하는 클래스 : **상위 클래스, parent class, base class, super class**
- ✓ 상속 받는 클래스 : **하위 클래스, child class, derived class, subclass**

4. 상속문법

Class B **extends** A {

}

상속

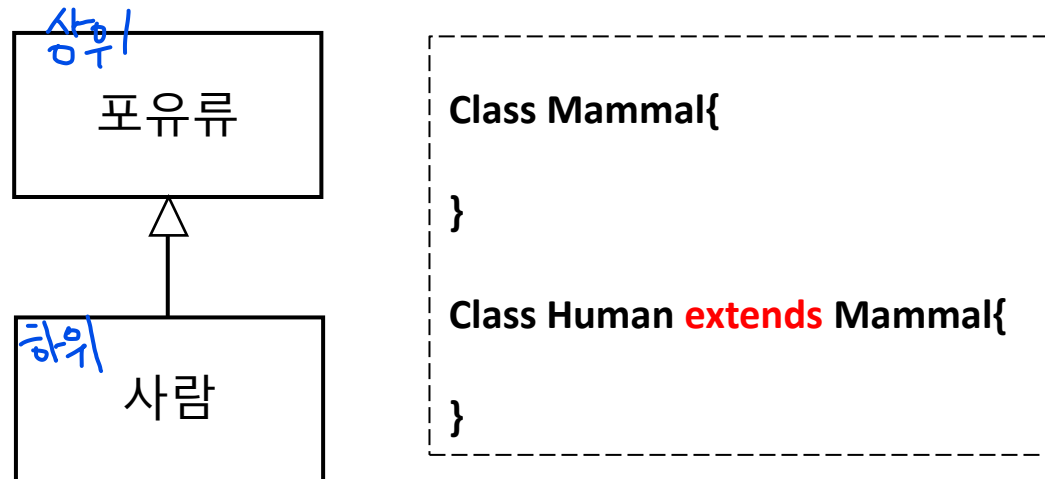


A 클래스가
B 클래스에게
상속한다
=
B 클래스가
A 클래스를
상속받는다.



1. 상속이란?

- ✓ 상위 클래스는 하위 클래스 보다 일반적인 의미를 가짐
- ✓ 하위 클래스는 상위 클래스 보다 구체적인 의미를 가짐

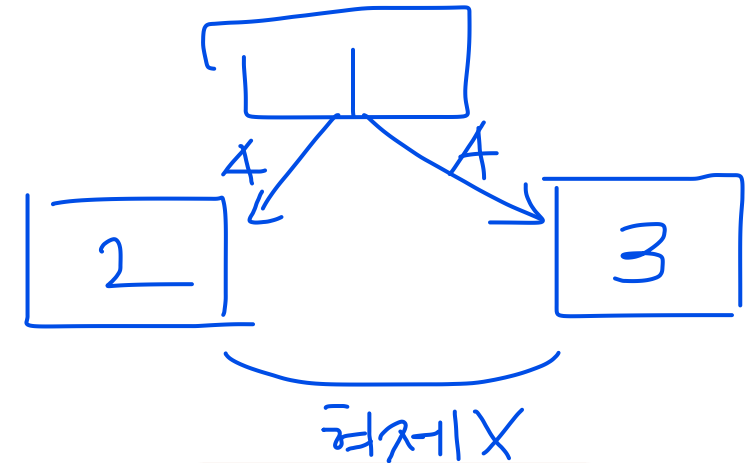
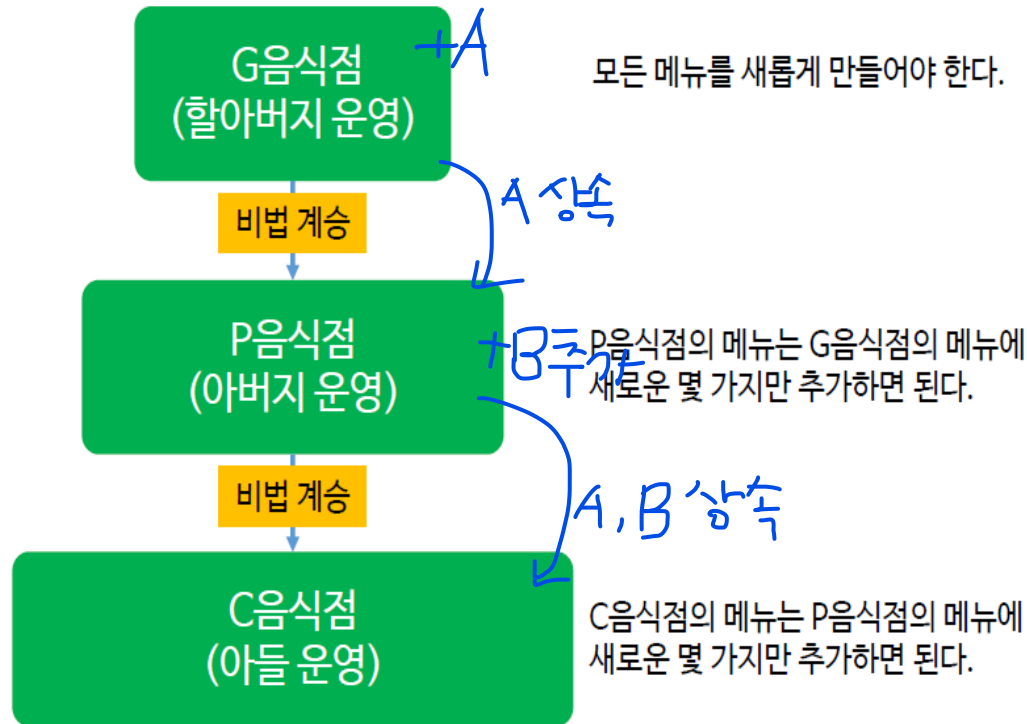


Extends 뒤에는 **단 하나의 class 만 사용할 수 있음**

자바는 **single inheritance**만을 지원 함.



- 기존에 검증된 class를 이용해서 빠르고 쉽게 새로운 class를 만들 수 있다.



검증된 메뉴를 이용해서 쉽고 빠르게 개업할 수 있다.



- Extends 키워드를 이용해서 상속을 구현한다.
- 하위 클래스가 생성될 때 항상 **상위 클래스**가 먼저 생성된다.
- 인스턴스를 함과 동시에 기본 생성자를 불러온다.

자바의 상속은 단일상속만 가능하다.

```
package lec18Pjt001;

public class ParentClass {

    public ParentClass() {
        System.out.println("ParentClass constructor");
    }

    public void parentFun() {
        System.out.println("-- parentFun() START -- ");
    }
}
```

상속

```
package lec18Pjt001;

public class ChildClass extends ParentClass {

    public ChildClass() {
        System.out.println("ChildClass constructor");
    }

    public void childFun() {
        System.out.println("-- childFun() START -- ");
    }
}
```

```
package inheritance;

public class ParentMain {

    public static void main(String[] args) {

        ChildClass child = new ChildClass();
        child.Chidfun();
    }
}
```

ParentClass Constructor
ChildClass Constructor
Childfun START



- ChildClass 가 ParentClass 를 상속 받았기 때문에 ParentClass의 메서드를 불러올 수 있다.

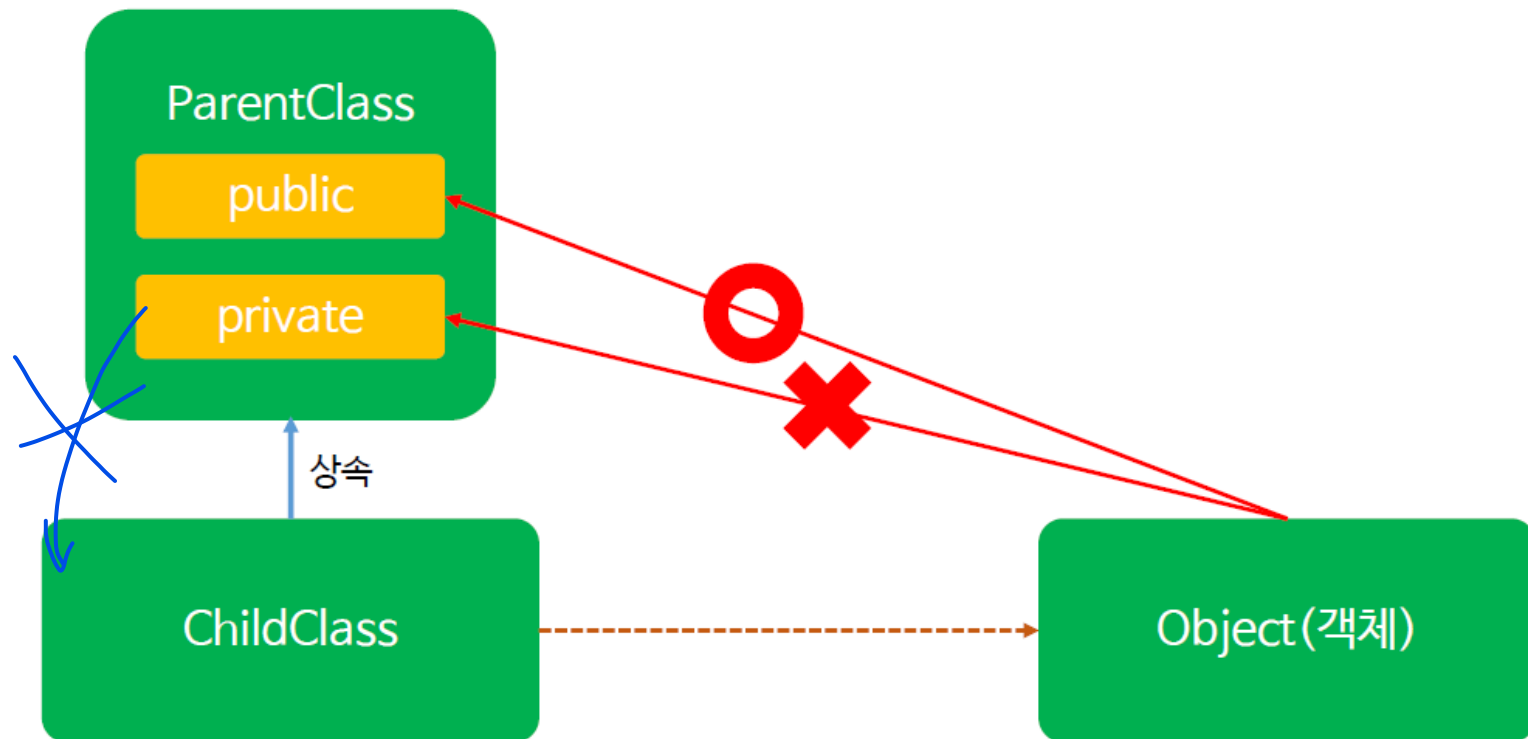
```
package inheritance;  
  
public class ParentMain {  
  
    public static void main(String[] args) {  
  
        ChildClass child = new ChildClass();  
        child.Parentfun();  
  
    }  
}
```



```
ParentClass Constructor  
ChildClass Constructor  
Parentfun START
```



- 자식 클래스는 부모 클래스의 모든 자원을 사용할 수 있지만, **private** 접근 제어자의 속성과 메서드는 사용할 수 없다.
- 단, private는 생성이 안되는게 아니라 접근만 못한다는 사실을 유의하자





- ParentClass의 variable a, variable b의 접근제어자를 **private**로 작성시 ChildClass에서 접근 할 수 없음을 주의하자. 단, private 는 생성이 안되는게 아니라 접근만 못한 다는 것에 주의하자
- 이럴 땐 **Protected** 접근제어자를 사용하자.

```
package inheritance;

public class ParentClass {

    private int a;
    private int b;

    public ParentClass() {
        System.out.println("ParentClass Constructor");
    }

    public void Parentfun() {
        System.out.println("Parentfun START");
    }
}
```

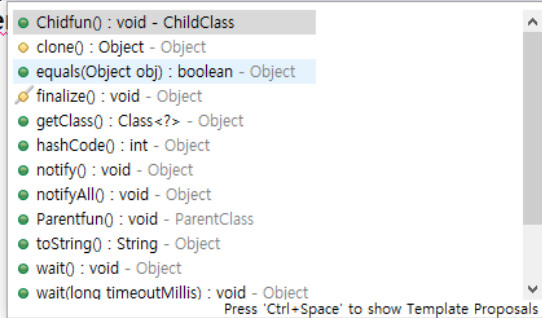


```
package inheritance;

public class ChildClass extends ParentClass{

    public ChildClass() {
        System.out.println("ChildClass Constructor");
    }

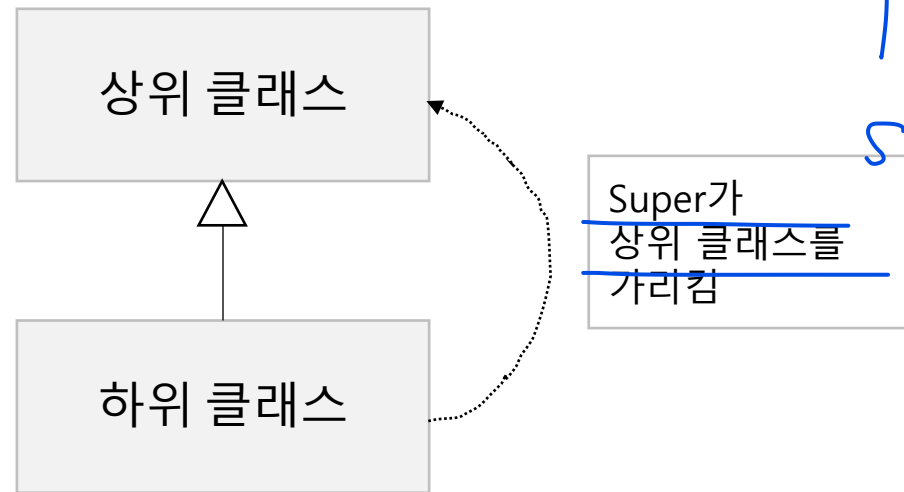
    public void Chidfun() {
        this.
        System
    }
}
```



Chidfun() : void - ChildClass
clone() : Object - Object
equals(Object obj) : boolean - Object
finalize() : void - Object
getClass() : Class<?> - Object
hashCode() : int - Object
notify() : void - Object
notifyAll() : void - Object
Parentfun() : void - ParentClass
toString() : String - Object
wait() : void - Object
wait(long timeoutMillis) : void - Object
Press 'Ctrl+Space' to show Template Proposals



- this 가 자기 자신의 인스턴스의 주소를 가지는 것 처럼 **super**는 하위 클래스가 상위 클래스에 대한 주소를 가지게 된다.
- 하위 클래스가 상위 클래스에 접근 할 때 사용할 수 있다.



this = 나
super = 부모



1. **Super**는 자신이 상속받은 부모 클래스에 대한 레퍼런스 변수로, **부모 클래스의 멤버에 접근할 때 사용**된다.
 - ✓ 주로 객체안에 있는 부모의 멤버변수와 자신의 멤버변수를 구별하기 위해 사용된다.
2. **Super()**는 자식 클래스의 생성자에서 **부모 클래스의 생성자를 호출하기 위해서 사용**된다.
 - ✓ **super()**는 생성자 코드안에서 사용 될 때, 다른 코드에 앞서 **첫줄에 사용**되어야 한다.
 - ✓ 자식 클래스의 모든 생성자는 부모 클래스의 생성자를 포함하고 있어야 합니다. 그런데 만약 **자식 클래스의 생성자에 부모 클래스의 생성자가 지정되어 있지 않다면, 컴파일러가 자동으로 부모 클래스의 기본생성자를 호출**합니다.
(이러한 경우, 부모클래스에 매개변수가 있는 생성자만 있고, 기본생성자가 없어 기본생성자를 호출할 수 없다면 에러가 발생합니다.)

부모 클래스

```
public class Book {
    int price = -1;

    public Book(int price) {
        this.price = price;
    }
}
```

자식 클래스

```
public class EnglishBook extends Book {
    public EnglishBook() { }
}
```

Error 발생

- 생성자안에 부모클래스의 생성자가 없으므로
- 자동으로 부모클래스의 기본생성자를 호출하나
- 부모클래스에 기본 생성자가 없으므로 에러 발생



- 상위 클래스를 호출할 때 super 키워드를 이용한다.

```
ParentClass
int openYear = 1995;

public void makeJJajang() {
    System.out.println(" -- makeJJajang() START --");
}
```

상속

```
ChildClass
int openYear = 2000;

public void getOpenYear() {
    System.out.println("ChildClass's Open year : " + this.openYear);
    System.out.println("ParentClass's Open year : " + super.openYear);
}
```

```
// super
ChildClass c = new ChildClass();
c.getOpenYear();
```



```
ParentClass constructor
ChildClass constructor
ChildClass's Open year : 2000
ParentClass's Open year : 1995
```



기존의 클래스를 재사용하여 새로운 클래스를 작성하는 것을 **상속**이라 한다.

쉽게 말하자면, 부모 클래스의 멤버를 자식 클래스에게 물려주는 것이다.

상속은 이미 잘 개발된 클래스를 재사용해서 새로운 클래스를 만들기 때문에 코드의 중복을 줄여준다.

보다 적은 양의 코드로 새로운 클래스를 작성할 수 있고, 코드를 공통적으로 관리할 수 있기 때문에 코드의 추가 및 변경이 매우 용이하다

자바에서 상속을 구현하는 방법은 새로 작성하고자 하는 클래스의 이름 뒤에 상속받고자 하는 클래스의 이름을 '**키워드 extends**'와 함께 써 주기만 하면 된다.

```
1 class A {  
2     int field;  
3     void method() { ... }  
4 }  
5  
6 class B extends A {  
7     String field2;  
8     void method2() { ... }  
9 }
```

위 두 클래스는 서로 상속 관계에 있다고 하며, 상속해주는 클래스를 '**조상(부모, 상위, 기반) 클래스**'라 하고 상속 받는 클래스를 '**자손(자식, 하위, 파생) 클래스**'라 한다.

여기서 하나 주의할 점은 **자바는 다중 상속을 지원하지 않는다는 것**이다.



```
1 class Parent {  
2     int age;  
3 }  
4  
5 class Child extends Parent {  
6     void play() {  
7         System.out.println("놀자~");  
8     }  
9 }
```

부모에게 age라는 멤버 필드를 상속 받았지만, 자신이 가지고 있는 play() 메소드는 부모 클래스에게 아무런 영향도 미치지 못한다.

부모 클래스가 변경되면 자식 클래스는 자동적으로 영향을 받게 되지만, 자식 클래스가 변경되는 것은 부모 클래스에 아무런 영향을 주지 못한다.

자식 클래스는 부모 클래스의 모든 멤버를 상속 받으므로 항상 조상 클래스보다 같거나 많은 멤버를 갖는다.

상속에 상속을 거듭할수록 상속받는 클래스의 멤버 개수는 점점 늘어나게 된다.

생성자와 초기화 블록은 상속되지 않는다. 멤버만 상속된다.

자식 클래스의 멤버 개수는 조상 클래스보다 항상 같거나 많다.



```
1 class Parent { }  
2 class Child extends Parent { }  
3 class Child2 extends Parent { }
```

위 코드를 보면, Child 클래스와 Child2 클래스가 Parent 라는 같은 클래스를 상속받아 형제 관계로 볼 수 있는데 클래스 간의 관계에서 형제 관계는 없다.

부모와 자식의 관계(상속 관계)만 존재한다는 것을 명심하자.

Child와 Child2 클래스에 공통으로 추가해야할 내용이 생긴다면 Parent 클래스에 추가하면 된다.

상속 관계이기 때문에 Parent 클래스의 기능을 사용할 수 있기 때문에 공통 기능 관리가 편해진다.

이처럼 같은 내용의 코드를 하나 이상의 클래스에 중복적으로 추가해야하는 경우에는 상속 관계를 잘 이용해서 코드의 중복을 최소화할 수 있다.

```
1 class Parent { }  
2 class Child extends Parent { }  
3 class Child2 extends Parent { }  
4 class GrandChild extends Child { } cs
```

자손 클래스는 조상 클래스의 모든 멤버를 물려받으므로 GrandChild클래스는 Child클래스의 모든 멤버, Child 클래스의 조상인 Parent클래스의 모든 멤버까지 상속받게 된다.

그래서 GrandChild클래스는 Child클래스의 자손이면서 Parent클래스의 자손이기도 하다.

좀 더 자세히 말하자면, Child클래스는 GrandChild클래스의 직접 조상이고, Parent클래스는 GrandChild클래스의 간접 조상이 된다.

그래서 GrandChild클래스는 Parent클래스와 간접적인 상속관계에 있다고 할 수 있다.



```
1 class Parent {  
2     int age;  
3 }  
4 class Child extends Parent { }  
5 class Child2 extends Parent { }  
6 class GrandChild extends Child { }
```

추가된 멤버 변수 age는 Parent클래스의 모든 자손에 추가된다.

반대로 멤버 변수 age를 제거한다면, Parent의 자손 클래스인 Child, Child2, GrandChild에서도 제거된다.

이처럼 조상 클래스만 변경해도 모든 자손 클래스에, 자손의 자손 클래스까지 영향을 미치기 때문에, 클래스 간의 상속 관계를 맺어 주면 자손 클래스들의 공통적인 부분은 조상 클래스에서 관리하고 자손 클래스는 자신에 정의된 멤버들만 관리하면 되므로 각 클래스의 코드가 적어져서 관리가 쉬워진다.

연습 문제





다음과 같은 클래스를 이용하여 문제1>을 풀어보세요.(단, 패키지명은 inheritance로 지정)

```
package inheritance3;

public class TV {
    protected int size; //인치

    public TV() {}
    public TV(int size) {
        this.size = size;
    }

    public int getSize() {
        return size;
    }

    public void setSize(int size)
    {
        this.size = size;
    }
}
```

문제1> 다음 main() 메소드와 실행 결과를 참고하여 TV를 상속받은 ColorTV 클래스를 작성하라.

```
public class Main {

    public static void main(String[] args) {
        ColorTV myTV = new ColorTV(32, 1024);
        myTV.printProperty();
    }

}
```



32인치 1024컬러



다음과 같은 클래스를 이용하여 문제2>을 풀어보세요. (단, 패키지명은 inheritance02로 지정)

```
package inheritance3;

public class TV {
    protected int size; //인치

    public TV() {}
    public TV(int size) {
        this.size = size;
    }

    public int getSize() {
        return size;
    }

    public void setSize(int size)
    {
        this.size = size;
    }
}
```

문제2> 다음 main() 메소드와 실행 결과를 참고하여 ColorTV를 상속받는 IPTV 클래스를 작성하라.

```
public class Main {

    public static void main(String[] args) {
        IPTV iptv = new IPTV("192.1.1.2", 32, 2048);
        // "192.1.1.2" 주소에 32인치, 2048컬러
        iptv.printProperty();
    }
}
```



나의 IPTV는 192.1.1.2주소의 32인치 2048컬러



Package 명 inheritance03 로 작성 할 것

Class 명 : ShapPencil, BollPen, FounctionPen, Main

문제3> 아래와 같이 출력되도록 작성 하시오.

<멤버변수>

Residual : 잔여량

thickness : 굵기

Name : 펜이름

<출력 결과 화면>

ShapPencil: 잔여량 :10, 펜굵기 :1

Ball: 잔여량 :25, 펜굵기 :2, 색상:black

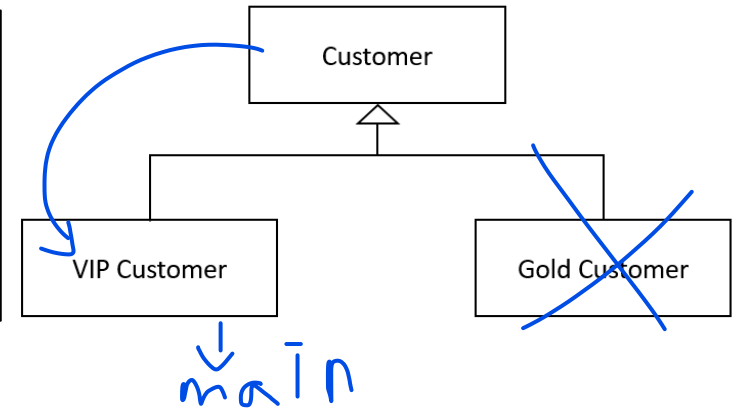
FunctionPen: 잔여량 :15, 펜굵기 :2, 색상:blue



문제4> 아래 조건에 만족하는 프로젝트를 작성하시오. (단, 패키지명은 inheritance04로 지정)

<조건>

- 고객의 정보를 활용하여 고객 맞춤 서비스를 구현
- 고객의 등급에 따라 차별화 된 할인율과 포인트를 지급
- 등급에 따른 클래스를 따로 구현하는 것이 아닌 일반적인 클래스를 먼저 구현하고 그 보다 기능이 많은 클래스는 상속을 활용하여 구현한다.



Customer 클래스 속성

멤버 변수	설명
customerID	고객 아이디
customerName	고객 이름
customerGrade	고객 등급기본 생성자에서 지정되는 기본 등급은 SILVER 입니다.
bonusPoint	고객의 보너스 포인트- 고객이 제품을 구매할 경우 누적되는 보너스 포인트입니다.
bonusRatio	보너스 포인트 적립 비율- 고객이 제품을 구매할 때 구매 금액의 일정 비율이 보너스 포인트로 적립됩니다. 이때 계산되는 적립 비율입니다.- 기본 생성자에서 지정되는 적립 비율은 1%입니다. 즉, 10,000원을 사면 100원이 적립됩니다.



문제4> 아래 조건에 만족하는 프로젝트를 작성하시오.

Customer 클래스 메서드

메서드	설명
Customer()	기본 생성자고객 한 명이 새로 생성되면 customerGrade는 SILVER 이고, bonusRatio는 1%로 지정함
calcPrice(int price)	제품에 대해 지불해야 하는 금액을 계산하여 반환함 할인이 없는 경우 가격을 그대로 반환함 가격에 대해 보너스 포인트 비율을 적용하여 보너스 포인트를 적립
showCustomerInfo()	고객 정보를 출력함 고객 이름, 등급, 현재 적립된 포인트 정보를 보여 줌

VIPCustomer 클래스

단골 고객에 대한 혜택이 필요함

우수 고객을 관리하기 위해 다음과 같은 혜택을 줌

1. 고객 등급 : VIP
2. 제품 구매 할인율 : 10%
3. 보너스 포인트 : 5%
4. 담당 전문 상담원 배정



<문제4번의 출력 결과물>

지불 금액은 10,000원이고,이성계님의 등급은 SILVER이며, 보너스 포인트는 100입니다.
지불 금액은 9,000원이고,김길동님의 등급은 VIP이며, 보너스 포인트는 500입니다., 담당
전문 상담은 1010번입니다.